

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Dashboard 'The Power Grid' (SCRUM-25)

1. CONTEXTO

Se requiere un dashboard interactivo de simulación de valor con temática cyberpunk para presentaciones de preventa. El requerimiento técnico ya define explícitamente el stack: HTML5 semántico, CSS3 puro y Vanilla JavaScript ES6+. No hay backend, no hay persistencia de datos, no hay autenticación. Es una Single Page Application (SPA) estática de propósito demostrativo.

2. DECISIÓN: Aplicación Estática de Archivo Único (Single-File SPA)

Se adopta una arquitectura de **aplicación web estática sin bundler, sin framework y sin servidor**. La entrega final es un conjunto mínimo de archivos estáticos (HTML + CSS + JS) que pueden abrirse directamente en un navegador o servirse desde cualquier CDN/servidor de archivos estáticos (Nginx, GitHub Pages, S3).

Estructura modular dentro del JavaScript: se separan responsabilidades en módulos ES6 nativos (sin Webpack/Vite) organizados por dominio funcional: `state`, `renderer`, `kpi-engine`, `canvas-engine`, `ui-controller`.

3. JUSTIFICACIÓN BASADA EN YAGNI Y KISS

- **¿Por qué NO React/Vue/Angular?** El requerimiento prohíbe frameworks de UI externos explícitamente. Además, YAGNI: no hay gestión de estado compleja que justifique un Virtual DOM. El estado global es un único objeto JS plano con 6 booleanos y valores numéricos.
- **¿Por qué NO Vite/Webpack?** La aplicación no tiene dependencias de terceros que requieran resolución de módulos en build-time. Los módulos ES6 nativos (`type='module'`) son suficientes y eliminan la necesidad de toolchain de build.
- **¿Por qué NO un backend/API?** Todos los datos son simulados y configurados en `KPI\_CONFIG`. No existe fuente de datos externa. Agregar un servidor sería BDUF puro.
- **¿Por qué NO Canvas único?** El requerimiento especifica explícitamente dos canvas superpuestos (`particleCanvas` + `connectionCanvas`) por razones de rendimiento de repintado selectivo. Se respeta esta decisión de diseño.
- **¿Por qué NO TypeScript?** El requerimiento dice Vanilla JS ES6+. Agregar un paso de compilación para una app de demostración sin equipo de mantenimiento a largo plazo viola KISS.
- **Componentes externos: 0 (CERO)**. No hay base de datos, no hay caché, no hay cola de mensajes. Cumple el criterio de máximo 2 dependencias externas con margen.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo de mantenibilidad**: Sin TypeScript, los errores de tipo en `KPI\_CONFIG` solo se detectan en runtime. Mitigación: JSDoc en funciones críticas.
- **Riesgo de compatibilidad**: El uso de ES6 modules nativos requiere un servidor HTTP para desarrollo local (no funciona con `file://` en Chrome por CORS). Mitigación: documentar en README el uso de `python -m http.server` o Live Server de VSCode.
- **Riesgo de rendimiento en dispositivos bajos**: Dos canvas + requestAnimationFrame a 60fps puede ser pesado en hardware antiguo. Mitigación: el modo Overdrive es opt-in y el bucle de partículas tiene un cap de cantidad máxima.
- **Sin tests automatizados**: La lógica de KPI es pura (input !' output) y puede testearse manualmente. Para una app de demo de preventa, el costo de configurar Jest supera el beneficio.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1
- src/styles/main.css | Estado: APROBADO | Iteraciones: 2
- src/styles/components.css | Estado: APROBADO | Iteraciones: 1
- src/styles/animations.css | Estado: APROBADO | Iteraciones: 1
- src/js/state.js | Estado: APROBADO | Iteraciones: 2
- src/js/kpi-engine.js | Estado: APROBADO | Iteraciones: 2
- src/js/canvas-engine.js | Estado: APROBADO | Iteraciones: 1
- src/js/ui-controller.js | Estado: APROBADO | Iteraciones: 1
- src/js/main.js | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 145,039

Tokens de salida (output): 60,585

Total tokens: 205,624

Horas-hombre estimadas: ~21 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,749 input | !"1,444 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"21,269 input | !"14,958 output | 10 llamadas

FRONTEND-DESARROLLO: !"42,705 input | !"36,911 output | 13 llamadas

QA-VALIDACION: !"78,316 input | !"7,272 output | 13 llamadas